

Module 5: Lesson 1

GRUs and LSTM



Outline

- ▶ Gated Recurrent Units (GRUs)
- ▶ Reset and Update gates in GRUs
- ▶ Long Short-Term Memory (LSTM)
- ▶ Input, Forget, and Output gates in LSTM

Gated Recurrent Units (GRUs)

One inherent problem that we encountered in standard RNN models is that long sequences can lead to numerical instability and vanishing and exploding gradients.

In practice, there are some pros and cons of using long sequences for prediction tasks, but what we would ideally like is to have flexibility regarding the length and relevancy of the sequence.

For instance, we may face the following situations:

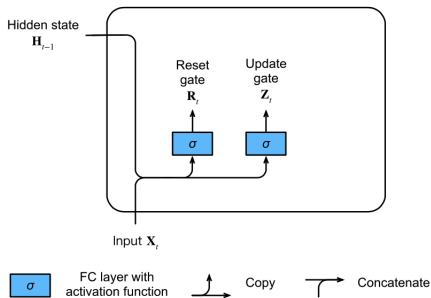
- ▶ An early observation may be highly relevant to explain all future observations → **memory cell?**
- ▶ Some tokens are not relevant → **skip these tokens?**
- ▶ There are logical breaks/parts of sequences (e.g., book chapter) → **reset internal state representation?**

A number of methods have been used to account for these desirable actions. One of the earliest is Long Short-Term Memory (LSTM). We will see this one afterwards and, for now, focus on another simpler one: Gated Recurrent Units (GRUs).

Reset and Update gates in GRUs

The main distinction between vanilla RNNs and GRUs is the fact that the latter incorporate mechanisms to let the network decide when a hidden state should be **updated** or **reset**.

- This is possible thanks to **Reset** and **Update** gates:
 - A reset gate allows us to control how much of the previous state we still want to remember.
 - An update gate allows us to control how much of the new state is a copy of the old one.



Reset and Update gates in GRUs

Mathematically, for a given time step t , minibatch of inputs X_t , and hidden state denoted by H_t , we can mathematically define reset and update gates as:

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

Note that activation functions σ are sigmoids, as we want to transform input to values in $(0, 1)$ interval.

The incorporation of these gates into our models will produce a **candidate hidden state** ($\tilde{H}_t$) which moderates the influence that previous hidden states have on new ones:

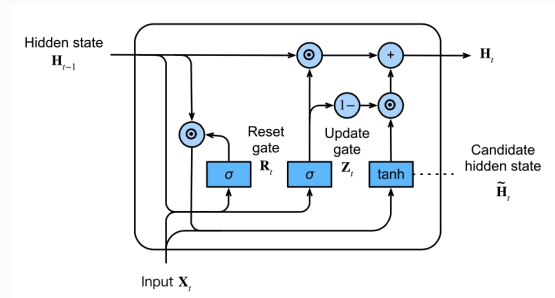
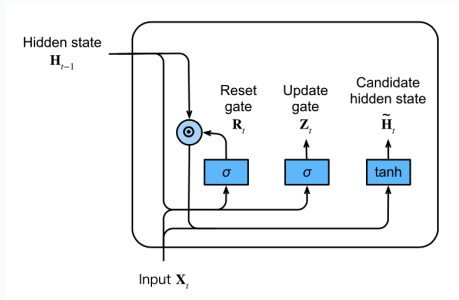
$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

Finally, we can determine our **hidden state** (H_t) by using the candidate one:

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

Reset and Update gates in GRUs

Here you have a graphical depiction (section 10.1 in the book) of how reset and update gates work:



Note how two interesting things will happen here:

- ▶ When R_t is close to 0 (1), the candidate hidden state is the result of an MLP (vanilla RNN).
- ▶ As Z_t is close to 1 (0), the hidden state approaches the old state (the candidate).

Long Short-Term Memory (LSTM)

In GRUs, reset gates capture short-term dependencies in sequences, while update gates capture long-term ones. LSTM captures these dependencies in a slightly more complex way.

- ▶ An LSTM is ruled by a **Memory cell** (or cell) that stores the additional information and has the same shape of hidden state.
- ▶ The memory cell is governed by a number of gates:
 - **Output gate**: Reads the entries from the cell.
 - **Input gate**: Decides when to read data into the cell.
 - **Forget gate**: Mechanism to reset the content of the cell.

As with GRUs, these gates decide when to remember and when to ignore the inputs in the hidden state. Mathematically:

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$$

Input, Forget, and Output gates in LSTM

In implementing these gates in our LSTM, we need to first design our memory cell, which will be governed by these gates that decide how much information is remembered, how much is forgotten, and how much actually passes through to the hidden state.

These decisions depend on the actions of the various gates to arrive at the hidden state:

- First, we define a **candidate memory cell** $\tilde{C}_t$:

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

- Second, given the results from the forget and input gates, the final memory cell form will be:

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

- Finally, we can compute the hidden state taking into account the output gate:

$$H_t = O_t \odot \tanh(C_t)$$

Input, Forget, and Output gates in LSTM

The following picture illustrates how input, forget, and output gates work:

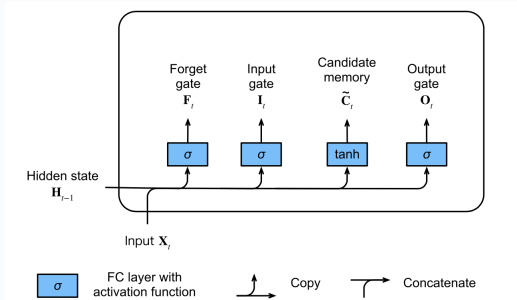


Fig. 10.2.2: Computing the candidate memory cell in an LSTM model.

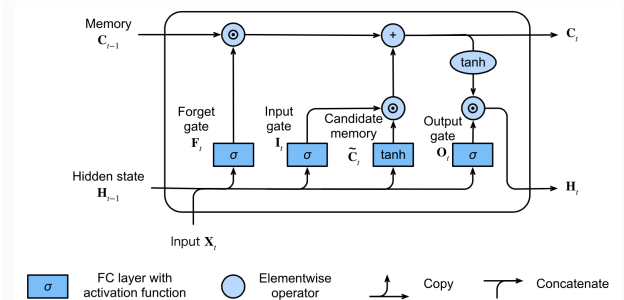


Fig. 10.2.4: Computing the hidden state in an LSTM model.

Summary of Lesson 1

In Lesson 1, we have:

- ▶ Introduced GRUs and LSTM as modern recurrent neural networks
- ▶ Discussed different types of gates to introduce memory (or lack of it) in our models
- ▶ Unveiled the mathematical way to express the functioning of these gates in both GRUs and LSTM.

⇒ **References for this lesson:**

Chapter 10, Sections 10.1 and 10.2 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice by investigating the power of LSTM.